

Serviços e Interfaces para a Cloud - SIC_TSIW

UNIDADE CURRICULAR

Running microservices with Docker containers

FICHA

Group work: This exercise should be completed in groups

Deadline: Submit on Moodle until 23h:59 of 17/11/2025

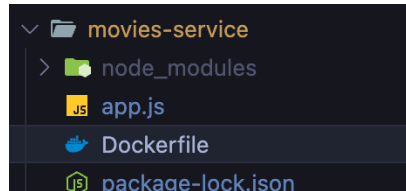
In this exercise, you'll run your microservices application using **Docker containers**, which is a more lightweight and efficient way of deploying applications compared to virtual machines. Docker allows you to package an application with all its dependencies into a container, ensuring consistent behavior across different environments. This exercise will help you understand containerization, service communication, and database integration in a microservices architecture.

1. Create a Dockerfile and build a docker image for each service and run the services in a Docker container.
2. Guarantee services are able to communicate with each other between containers.
3. Add a real database to each service by running it in a container.

You should submit your code and a screenshot of the docker ps command output after you have everything running and working.

How to build a docker image of a NodeJS service

Create a Docker file for each service:



```
# Use Node 20 as the base image from Docker Hub
FROM node:20

# Set working directory, where your app code will be copied
WORKDIR /app

# Copy package.json and install dependencies
COPY package*.json ./
RUN npm install

# Copy the rest of the app
COPY . .

# Expose the port the app runs on
EXPOSE 3001

# Command to run the app
CMD ["node", "app.js"]
```

After you created the Dockerfile you can build a Docker image from it:

```
docker build -t movies-service:1.0 .
```

Check if the docker image was created:

```
docker images
```

Now run a Docker container with the image created:

On mac/Linux:

```
docker run \
  --name movies-service \
  -p 3001:3001 \
  movies-service:1.0
```

On windows:

```
docker run `
  --name movies-service `
  -p 3001:3001 `
  movies-service:1.0
```

Do this for each service and try to access them via your browser.

How to enable communication between containers

You will notice that Movies Service has no connectivity to Reviews Service. To enable communication between Docker containers you need to create a network and add the containers to the same network. To create a Docker network do the following:

```
docker network create <your_network_name>
```

Now stop your application containers and rerun the services using:

On mac/Linux:

```
docker run -d \
  --name reviews-service \
  --network <your_network_name> \
  -p 3002:3002 \
  reviews-service:1.0
```

On windows:

```
docker run -d `
  --name reviews-service `
  --network <your_network_name> `
  -p 3002:3002 `
  reviews-service:1.0
```

Now your services can communicate with each other. However, when you use a docker network, **a service is no longer reachable by using localhost**. In Movies Service you need to update the address of Reviews Service to the name of the Docker container of Reviews Service:

Change it from:

```
axios.get(`http://localhost:3002/reviews?movieId=${movie.id}`);
```

To:

```
axios.get(`http://reviews-service:3002/reviews?movieId=${movie.id}`);
```

Once you changed the Reviews Service address in Movies Service, you need to **stop the Movies Service container, remove the stopped container and rebuild the Docker image of Movies Service again and run a new container.**

After these changes your containers should be able to communicate with each other, because they are in the same network.

How to run a database in a container and make the application containers communicate with it

You've been using hardcoded data for your services. Now let's use a real database, like MongoDB.

Run MongoDB in a container in the same network as your service containers:

In windows:

```
docker run -d `
  --name mongodb `
  --network <your_network_name> `
  -p 27017:27017 `
  -e MONGO_INITDB_ROOT_USERNAME=root `
  -e MONGO_INITDB_ROOT_PASSWORD=pass123 `
  mongo:8
```

In mac:

```
docker run -d \
  --name mongodb \
  --network <your_network_name> \
  -p 27017:27017 \
  -e MONGO_INITDB_ROOT_USERNAME=root \
  -e MONGO_INITDB_ROOT_PASSWORD=pass123 \
  mongo:8
```

Test your database connection using MongoDB compass in your computer.

You should stop and remove your Movies, Reviews and Users containers while you are testing the new code locally.

In your services, add a connection to MongoDB, create the models and change the logic to use a MongoDB database instead of local objects for movies, reviews and users. Do this in each service:

```
npm install mongoose
```

Example of the code changes in Movies Service:

```
...
const mongoose = require("mongoose");
...
mongoose.connect(
  "mongodb://root:pass123@localhost:27017/moviesdb?authSource=admin"
).then(() => console.log("MongoDB connected"))
.catch(err => console.error("MongoDB connection error:", err));

const Movie = mongoose.model("Movie", new mongoose.Schema({
  id: Number,
  title: String,
  year: Number
}));
```

While you are testing locally, you should use localhost for the connection to MongoDB:

```
"mongodb://root:pass123@localhost:27017/moviesdb?authSource=admin"
```

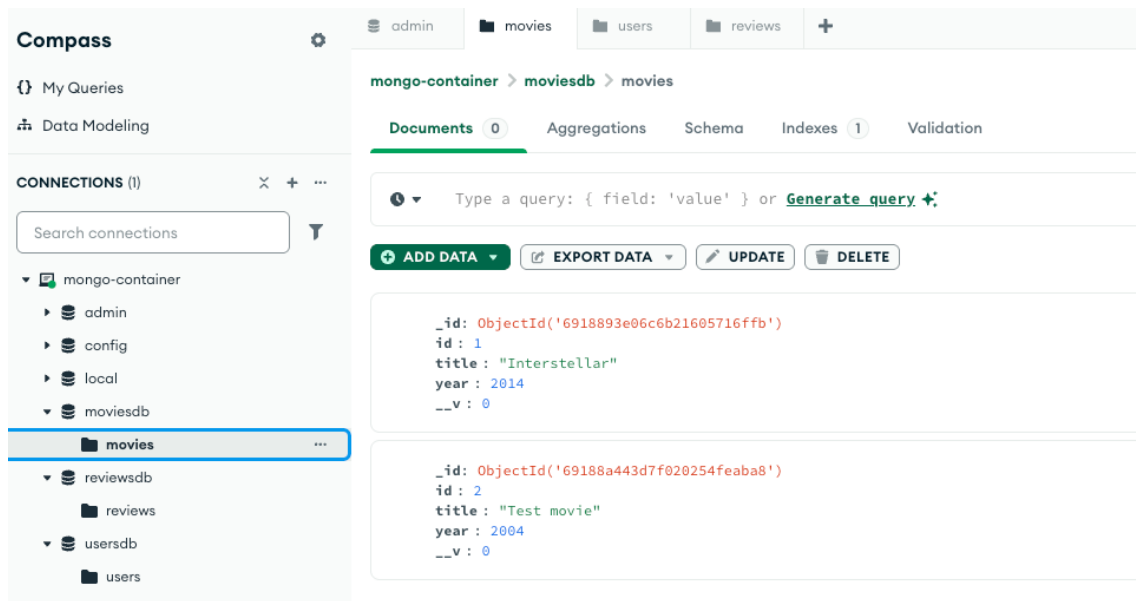
But once you tested everything and you want to run the services in a container, you should use the container name instead:

```
"mongodb://root:pass123@mongodb:27017/moviesdb?authSource=admin"
```

Example of interaction with MongoDB:

```
app.get("/movies", async (req, res) => {
  const movies = await Movie.find();
  res.json(movies);
});
```

You should have the following databases after integrating all services with MongoDB:



After you integrated your application with MongoDB, build the docker image again with the updated code and start a new container for each service, using the commands mentioned previously in this exercise.

All services should be able to communicate with each other and with MongoDB, while everything is running in a container.